

Win3D Project Proposal

Background and Introduction

Computer graphics are becoming more and more prevalent in the modern age, from the graphics you are looking at to read this paper to animation and CGI in films and TV, to graphics used in video games. 3D graphics in particular, are becoming more prevalent with the rise of 3D animation studios, and game studios. 3D graphics is a subset of computer graphics, where 3 dimensional objects, stored in a 3D vector space, have to be rendered onto a screen of just 2 dimensions. This comes with a host of new problems over traditional 2D graphics, and there have been lots of different solutions to these problems. There are different ways to store coordinates (cartesian or polar), different ways to handle rotation (euler matrices or quaternions), and different pipelines that transform object data into pixels on our screen (forward rendering vs deferred rendering).^[1]

With this has come the rise of the GPU - and the more modern GPGPU - a separate processor to the CPU that is specifically designed to handle massively parallel operations, such as graphics processing operations.^[2] Graphics APIs have also been developed to make easier the lives of programmers who are trying to render graphics to the screen. Graphics APIs and the GPU, unfortunately, abstract away a lot of what is necessary to render graphics to the screen, such as the graphics rendering pipeline. Moreover, the graphics rendering pipeline, includes stages that offer differing levels of customisability/programmability, dependent on the specific vendor.^[3] Some stages, such as the rasterization stage, offer no ability for it to be configured or programmed in any way; it has a set, fixed-function. Other stages offer varying levels of configurability, but you cannot directly program them. Therefore, my goal is to write my own graphics rendering system from scratch, on the CPU, without the use of the GPU or any graphics API.

Within 3D graphics rendering, there is an area called physically based rendering, where graphics rendering systems are designed to be accurate to physics and the real world. Physically based rendering systems must take into account physical laws such as the conservation of energy, etc. physically based rendering is often used for photo realistic rendering, which is what I will be using it for, but it is still capable of creating stylised effects. There is also the question of offline vs real-time rendering, where graphics are either rendered in real-time, as they are being shown to the customer; or beforehand, such that their quality can be increased, and shown to the customer later. This project will mainly focus on developing a physically-based, offline rendering system.^[4]

The Proposed Project

The main goal of this project is to develop a graphics API that implements the graphics rendering pipeline, and physically-based offline rendering techniques, such that users can develop and render high quality 3D graphics and scenes. Programmers using the API will be able to: create a “GraphicsPipeline” object composed of the different stages of the graphics pipeline; set different pipeline parameters such as the projection used; use it to pass imported objects through the pipeline; and inherit from it if they want to edit the pipeline in any way. The API should also have abstractions for high level application layer concepts like different objects (squares, spheres, illumination objects, viewports, etc) and shading techniques like raytracing, volumetric effects, caustic effects, etc. So in essence, it is an implementation of the graphics rendering pipeline with an API abstracting away each of its stages, including application layer abstractions, with the main purpose being for offline, physically based/photorealistic rendering.

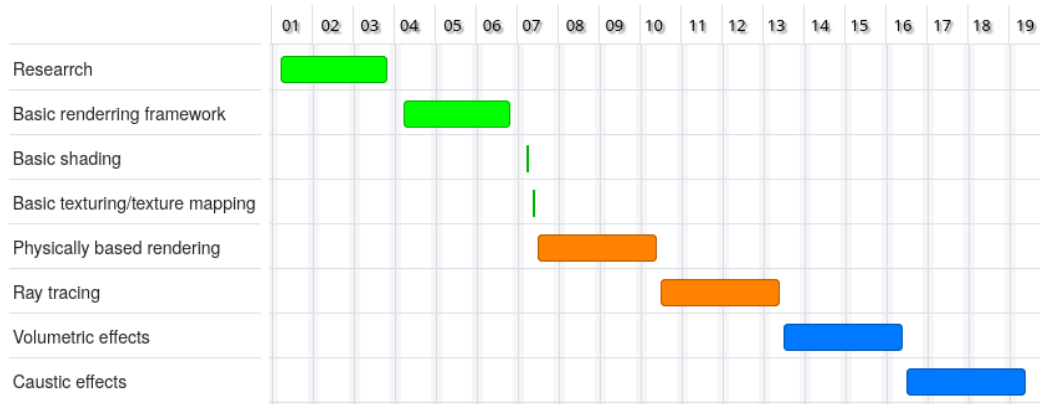
Project plan for implementation

Programme of work

- Research (3 weeks)
 - Research into the topic of 3D graphics, what will need to be implemented, and related topics.
- Basic rendering framework (3 weeks)
 - Includes the basic graphics rendering pipeline: the application phase; the geometry processing stage; the rasterization stage; and the pixel processing stage.
 - The ability to import objects from blender.
 - The ability to pass objects through the whole pipeline via a draw call.
- Basic shading (1 day)
 - Phong shading
- Basic texturing/texture mapping (1 day)
 - Can import and map textures to objects in the pixel processing stage of the graphics rendering pipeline
- Physically based rendering (3 weeks)
 - Implementations of various Bidirectional reflectance distribution functions
 - The ability to do physically based texturing
- Ray tracing (3 weeks)
 - Basic ray tracing
- Volumetric effects (3 weeks)
 - Effects for smoke, fire, clouds, etc
- Caustic effects (3 weeks)
 - Effects for glass, water, etc

Gantt chart

Green: Milestone 1; Orange: Milestone 2; Blue: Milestone 3.



References

- [1] GOMES, J. (2017) *DESIGN AND IMPLEMENTATION OF 3D GRAPHICS SYSTEMS*. First edition. Place of publication not identified: CRC Press.
- [2] Hwu, W. (2012) *GPU computing gems*. Jade ed. Boston, Mass: Morgan Kaufmann.
- [3] Akenine-Moller, T. *et al.* (2018) *Real-time rendering*. Fourth edition. Boca Raton: CRC Press.
- [4] Matt Pharr, Wenzel Jakob, and Greg Humphreys. (2016) *Physically Based Rendering: From Theory to Implementation* (3rd ed.). Morgan Kaufmann Publishers Inc.